Middleware

LESSON 4

A small piece of code in the path of every request

1 What is it?

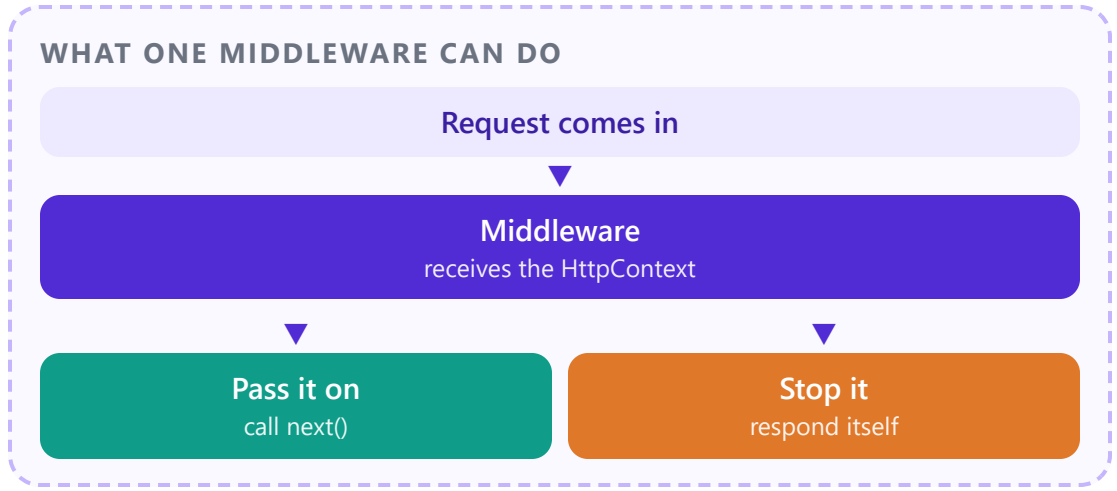
A small piece of code that stands in the way of the request. It receives the `HttpContext` and decides: do something and **pass the request to the next one**, or **stop the request** and respond itself.

2 Why does it exist?

Some work repeats with **every** request: redirecting http to https, handling errors, checking the user, logging requests. Writing it inside every function would repeat the code everywhere. A middleware gathers each piece of work in one place and applies it to everyone.

3 Who adds it, and when?

We add it in `Program.cs`, **after** `Build()` **and before** `Run()`, with `app.UseXxx()`. The Host builds the pipeline from them at startup.



The Request Pipeline

LESSON 4

An ordered chain of middlewares

1 What is it?

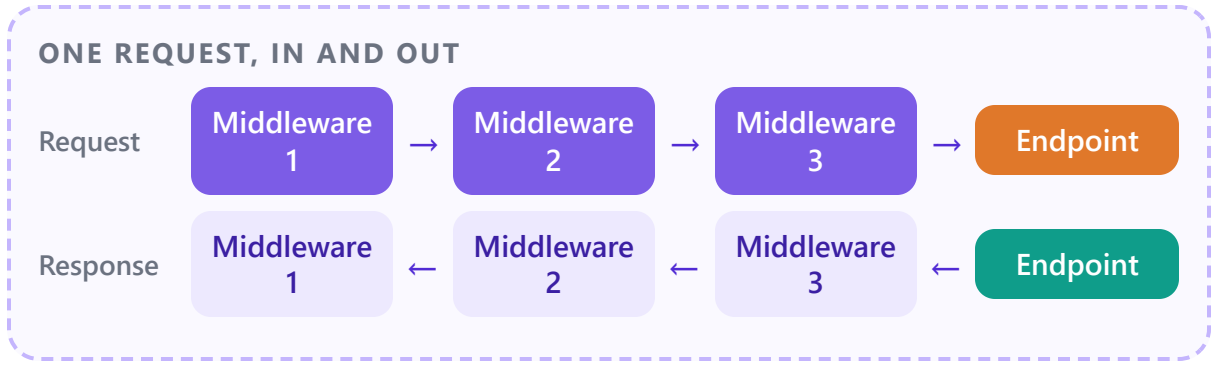
An **ordered chain** of middlewares. Every request passes through them one after another.

2 In which order?

The order is the order you **write them in** `Program.cs`. The request goes in, and the response comes back out through the same chain in **reverse order**, like the layers of an onion.

3 Two passes

Each middleware runs **twice**: once before it passes the request on, and once after the response comes back.



> The endpoint is the code that answers the request. It is at the **end** of the chain.

Experiment 1: your first middleware

LESSON 4

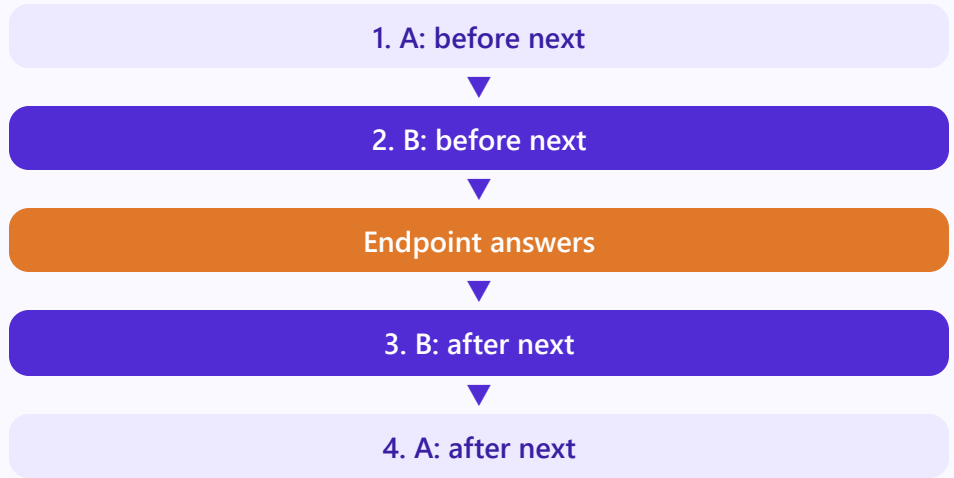
Code before `next()` runs on the way in. Code after it runs on the way out.

CODE

```
app.Use(async (context, next) =>
{
    Console.WriteLine("A: before next");
    await next();
    Console.WriteLine("A: after next");
});
```

- `context` is the `HttpContext`, the object that represents the current request and response.
- `next` passes the request to the next middleware.
- Add a second middleware in the same way and call it `B`.

THE ORDER OF THE MESSAGES



CONSOLE OUTPUT

```
A: before next
B: before next
B: after next
A: after next
```

> **Try it:** swap A and B and watch the output change. The **order matters**.

Experiment 2: stopping the request

LESSON 4

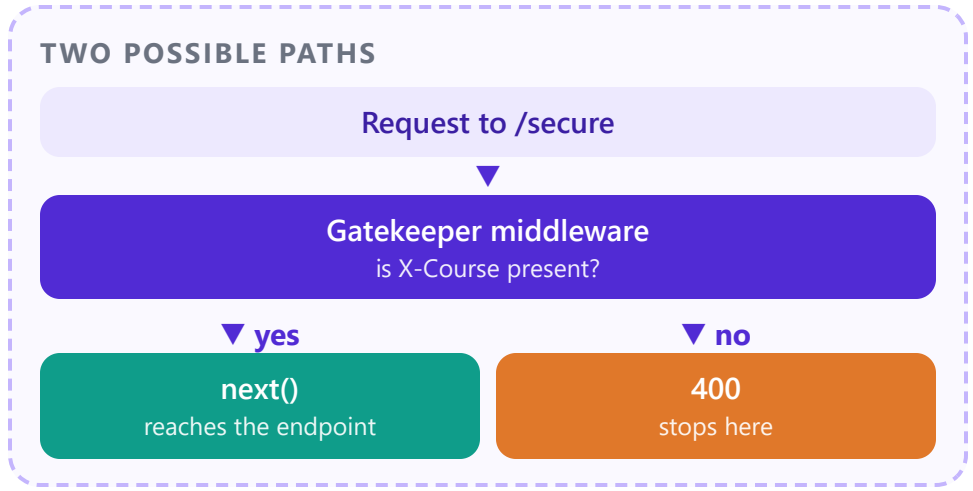
Short-circuit: respond without calling `next()`

What is a short-circuit?

The middleware responds **by itself**, without calling `next()`. The request never reaches what comes after. It works like a gatekeeper: if the request fails the check, it is rejected here and never reaches your code.

```
app.Use(async (context, next) =>
{
    if (context.Request.Path.StartsWithSegments("/secure") &&
        !context.Request.Headers.ContainsKey("X-Course"))
    {
        context.Response.StatusCode = StatusCodes.Status400BadRequest;
        await context.Response.WriteAsync("Missing X-Course header");
        return;          // we do not call next()
    }
    await next();
});
```

- Send the request **without** the header: `400 Bad Request` and the message.
- Send it **with** `X-Course: 1`: the request continues to the endpoint.
- The `return` without `next()` is what stops the request.

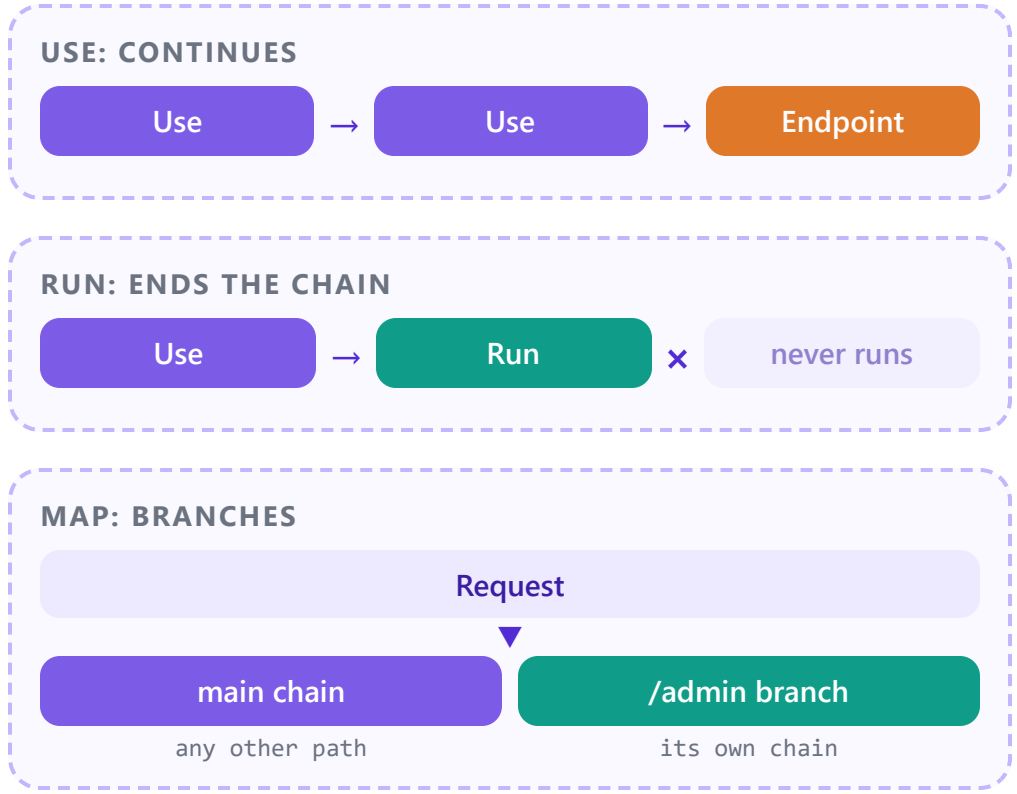


Use, Run and Map

LESSON 4

Three ways to add to the pipeline

Command	What it does	Passes on?
`app.Use(...)`	A normal middleware. It receives `next`.	Yes, if it calls `next()`
`app.Run(...)`	The **last** middleware. It has no `next`.	No. Anything after it never runs.
`app.Map("/path", ...)`	**Branches** the pipeline. Requests for this path go to a separate branch.	The branch has its own chain.



A MAP EXAMPLE

```
app.Map("/admin", adminApp =>
{
    adminApp.Run(async context =>
        await context.Response.WriteAsync("Admin branch"));
});
```

> Open `/admin` and you see **Admin branch**. An endpoint such as `MapGet` is also the end of a chain, because it is the code that answers the request.

Experiment 3: a middleware in its own class

LESSON 4

Keeps `Program.cs` clean, and the code reusable and testable

The shape

A class with two things: a **constructor** that receives `RequestDelegate`, and a method `InvokeAsync` that receives `HttpContext`.

```
public class RequestTimingMiddleware
{
    private readonly RequestDelegate _next;

    public RequestTimingMiddleware(RequestDelegate next) => _next = next;

    public async Task InvokeAsync(HttpContext context)
    {
        var watch = System.Diagnostics.Stopwatch.StartNew();
        await _next(context);
        watch.Stop();
        Console.WriteLine(
            $"{context.Request.Method} {context.Request.Path} " +
            $"took {watch.ElapsedMilliseconds} ms");
    }
}
```

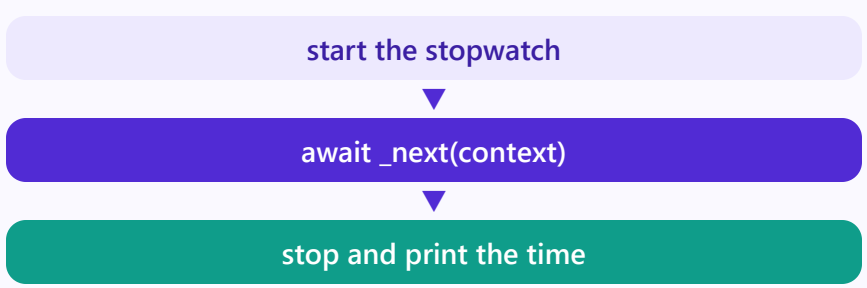
IN PROGRAM.CS

```
app.UseMiddleware<RequestTimingMiddleware>();
```

NOTES

- `_next` is the function that passes the request to the next middleware.
- The class is placed **below** `Program.cs` (after `app.Run()`) or in its own file.
- Console output: `GET / took 6 ms`.
- It measures the time of every request, which is very useful for logging.

AROUND THE NEXT MIDDLEWARE



Built-in middlewares and their usual order

LESSON 4

The recommended order, and why

#	Middleware	What it does
1	UseExceptionHandler	Catches errors from everything after it
2	UseHttpsRedirection	Redirects http to https
3	UseStaticFiles	Serves static files (images, CSS)
4	UseRouting	Chooses the endpoint for the request
5	UseCors	Lets other websites call the app
6	UseAuthentication	Who are you?
7	UseAuthorization	Do you have permission?
8	Endpoints	The code that answers the request

WHY THIS ORDER

UseExceptionHandler first, because it catches the errors of everything after it. If you put it last it catches nothing.

UseAuthentication before UseAuthorization, because you cannot check the permission of someone you do not know yet.

UseAuthorization before the endpoints, because permission is checked before your code runs.

In the modern minimal hosting style, `UseRouting` is added automatically, so we do not write it ourselves.

Summary

What to remember, and what to try

LESSON 4

SUMMARY

Concept	What it is
Middleware	A piece of code in the path of the request. It passes it on or stops it.
Pipeline	The chain of middlewares. Requests go in by order, and responses come out in reverse.
next()	Passes the request on. Not calling it stops the request (short-circuit).
Use / Run / Map	Normal / final / branch
Order	It matters. It decides what happens to every request.

TRY IT YOURSELF

- Write a middleware that adds a header named `X-Request-Id` to the response.
- Make the middleware that blocks requests without the header work on **all** paths, not only `/secure`, then test `/`.
- Swap the order of A and B and write down what changes.

REMEMBER THE PICTURE

